

Adaptive SLO-Aware Admission Control for Multi-Tenant Managed LLM Gateways

Taixing Bi
Independent Researcher
New York, NY, USA
taixingbijob@gmail.com

Abstract—Enterprise AI platforms increasingly centralize access to managed large-language-model (LLM) APIs behind multi-tenant gateways. These gateways can enforce quotas and latency service-level objectives (SLOs), but they do not observe provider-side queues, GPU occupancy, or instantaneous serving capacity. We study the resulting control problem using a production-inspired AWS Bedrock gateway with 100 logical tenants, heterogeneous priorities and TTFT SLOs, Redis-backed shared counters, and a real Amazon Nova Micro backend. A load sweep shows that the backend remains healthy at offered loads up to $20\times$ a configured gateway control budget, demonstrating that a static token budget can be a poor proxy for instantaneous managed-provider capacity. Under a constant-request-rate token burst, static SLO-aware admission reduces P99 TTFT but oversheds traffic, yielding only 4.6% effective SLO goodput. We introduce an adaptive SLO-aware controller that adjusts an effective capacity estimate using backend-health feedback and AIMD-style updates. It raises burst-phase effective SLO goodput to 84.4% while retaining 98.2% conditional SLO attainment and lower P99 TTFT than uncontrolled admission. A noisy-neighbor experiment reveals an important boundary: global capacity adaptation recovers goodput but weakens tenant isolation, indicating that capacity estimation and per-tenant protection are distinct control objectives. A small sensitivity study shows the goodput recovery persists across moderate AIMD parameter changes. These results motivate a two-level gateway design: adaptive provider-capacity estimation plus explicit tenant-isolation controls for opaque, elastic managed LLM services.

Index Terms—large language models, inference serving, admission control, multi-tenancy, SLO, AI gateway, Amazon Bedrock

I. INTRODUCTION

Large language models have shifted inference serving from a conventional request/response workload into a stateful, autoregressive service whose cost depends strongly on prompt length, generated length, batching state, and cache residency. Systems such as Orca, vLLM, FastServe, Sarathi-Serve, DistServe, Splitwise, Mooncake, and Llmunix improve execution efficiency through iteration-level scheduling, paged KV-cache management, preemption, chunked prefill, phase disaggregation, and runtime rescheduling [1]–[8]. These systems largely operate inside or immediately above an inference runtime, where GPU resources, request queues, and execution state are visible.

Enterprise deployments increasingly introduce a different layer: a centralized LLM gateway in front of managed providers such as Amazon Bedrock. The gateway authenticates callers, attributes usage to applications, applies tenant-specific

quotas, records cost and latency, and may reject or queue requests before they reach the provider. This pattern is itself a big-data-systems problem at enterprise scale: a single gateway operationalizes one foundation model behind heterogeneous, high-volume token streams from hundreds of internal applications, and must govern that shared resource the way a data platform governs shared storage or compute — through centralized quota, priority, and observability infrastructure rather than per-application configuration. Multi-tenant fairness work has shown that request-count rate limits are inadequate when clients have heterogeneous token costs [9], [10], while recent SLO-oriented serving systems exploit latency slack, deadlines, priorities, or predicted execution time to improve SLO attainment [11]–[15]. Token-pool abstractions similarly argue for admission at the API boundary using inference-native units [16].

Existing gateway- and pool-level token control largely treats platform capacity as a known quantity: an operator-configured entitlement, budget, or pool size that the controller allocates but does not question. That assumption is reasonable for a self-hosted cluster, where GPU count and batching policy are visible to the operator. It breaks down for a managed, multi-tenant provider such as Bedrock, where the enterprise-side budget is a governance artifact and the provider-side serving capacity is opaque and elastic, changing independently of any single client’s configuration. Our contribution is not the AIMD update rule itself, which is a classical technique [22]; it is the identification of this capacity-visibility gap at the gateway layer, an admission design that treats the token budget as an online estimate of a hidden, time-varying quantity rather than a fixed entitlement, and an empirical characterization of what that gains (goodput) and costs (tenant isolation) on a real managed backend.

Managed APIs create a control asymmetry that prior GPU-centric serving work does not address. A gateway sees request arrivals and observed responses, but not the provider’s internal queue depth, GPU allocation, batching policy, or instantaneous effective capacity. Provider quotas are documented in token- and request-based terms [25], [26], but a configured enterprise budget need not equal the capacity currently available behind an elastic managed service. Consequently, a static policy may either admit too aggressively when provider capacity contracts or shed too conservatively when provider capacity is plentiful.

This paper studies that mismatch empirically and asks: how

should a multi-tenant gateway protect heterogeneous TTFT SLOs when managed-provider capacity is opaque and elastic? We build a production-inspired gateway on AWS and compare no control, request-rate limiting, token windows, token bucket, priority-aware control, static SLO-aware admission, and a new adaptive SLO-aware policy. The adaptive controller treats its token budget as an estimate rather than a fixed truth and updates it using backend-health feedback with an additive-increase/multiplicative-decrease (AIMD) mechanism inspired by classical congestion avoidance [22].

Our contributions are:

- We characterize the mismatch between a configured gateway token budget and observed usable capacity of a real managed LLM backend, showing that token pressure is not equivalent to provider saturation.
- We design adaptive SLO-aware admission control that combines tenant entitlement, priority, SLO slack, token demand, and a dynamically updated effective-capacity estimate without requiring access to provider-side scheduling internals.
- We evaluate seven policies with 100 heterogeneous tenants under load sweep, constant-RPM token burst, and noisy-neighbor workloads, quantifying the latency-goodput-isolation tradeoff and identifying where adaptation helps and where per-tenant isolation remains necessary.

II. BACKGROUND AND RELATED WORK

A. LLM Inference Serving and SLOs

Early online prediction systems such as Clipper, Clockwork, Mark, InferLine, and INFaaS established the importance of tail-latency SLOs, adaptive batching, predictable execution, provisioning, and cost-aware cloud serving [17]–[21]. The broader distributed-systems literature likewise emphasizes that high-percentile latency dominates user experience at scale [23] and that scheduling delay itself can become material for short interactive jobs [24].

LLM inference adds variable-length prefill and autoregressive decode phases. Orca introduced iteration-level scheduling [1]; vLLM’s PagedAttention improved KV-cache memory efficiency [2]; FastServe applied preemptive scheduling [3]; Sarathi-Serve reduced throughput/latency interference with chunked prefill [4]; DistServe and Splitwise separated prefill and decode resources [5], [6]; Mooncake made KV cache a first-class schedulable resource [7]; and Llmunix used runtime migration to improve tail latency and isolation [8]. These systems assume visibility into execution resources that a managed-API gateway does not have.

B. Fairness, Priority, and Multi-Tenant Control

Virtual Token Counter (VTC) formalizes fairness using token-weighted service rather than request counts [9]. FairServe extends this direction using real multi-tenant Copilot traces and application-aware throttling [10]. SCORPIO, Niyama, SLOs-Serve, ProServe, and Cascade incorporate heterogeneous latency SLOs, deadline urgency, priority, or

latency budget into scheduling decisions [11]–[15]. Token Management in Multi-Tenant AI Inference Platforms moves the control boundary toward the API gateway and represents capacity with token throughput, KV cache, and concurrency entitlements [16].

How this paper differs. VTC, FairServe, and token-pool designs [9], [10], [16] share a structural assumption: platform capacity is an explicit, operator-set entitlement, and the control problem is dividing that known quantity fairly across tenants. That assumption holds when the operator runs the backend, but not when the backend is a managed, elastic, third-party service. Our setting keeps their fairness and priority mechanisms (tenant quotas, tiers, SLO slack) intact but adds a layer beneath them: the capacity those mechanisms divide is itself unknown and time-varying, observable only through externally visible signals (429s, TTFT, policy rejects) rather than through GPU or queue telemetry. The central challenge we study is therefore not only allocating a known capacity, but deciding how much capacity should be assumed at all — a problem orthogonal to, and composable with, prior fairness and priority mechanisms.

C. Feedback Control and Managed APIs

AIMD is a classical feedback-control mechanism that converges toward efficient resource use while reacting sharply to congestion [22]. We adapt this idea to gateway capacity estimation: healthy provider behavior increases the effective capacity estimate, while observed pressure contracts it. The analogy is intentionally limited. Unlike TCP, the gateway cannot observe a precise bottleneck queue, and provider elasticity can change independently of the client. We therefore use AIMD as a simple, interpretable controller rather than claiming a network-congestion model.

Amazon Bedrock exposes model-level quotas, token-burndown semantics, cross-Region inference, streaming inference, token counting, CloudWatch metrics, and retry guidance [25]–[32]. API Gateway also supports request throttling, but its quotas are best-effort and request-oriented [33]. These capabilities motivate a gateway layer that separates tenant policy from provider mechanics.

III. PROBLEM FORMULATION

We consider a gateway serving tenants $i \in \{1, \dots, N\}$. Tenant i has priority p_i , token entitlement q_i , and TTFT SLO d_i . Request r arrives at time a_r with estimated token demand t_r . The gateway observes queue waiting time w_r and maintains an estimate of backend TTFT, $\hat{L}(t)$. Static SLO-aware admission computes remaining slack $s_r = d_i - w_r - \hat{L}(t)$. Small slack increases urgency, while tenant entitlement and priority determine how scarce capacity is shared.

The difficulty is the platform capacity C . A self-hosted server can estimate C from GPU state, active sequences, KV-cache occupancy, or measured throughput. A managed API hides these variables. We distinguish a configured enterprise budget C_0 from instantaneous effective provider capacity $C^*(t)$. The gateway never observes $C^*(t)$ directly. A static

controller implicitly assumes $C^*(t) \approx C_0$; our experiments show this assumption can be badly conservative.

We evaluate two notions of SLO success. Conditional SLO attainment counts only admitted requests: $A_{cond} = |\text{served within SLO}|/|\text{admitted}|$. Effective SLO goodput uses all offered requests: $G_{eff} = |\text{served within SLO}|/|\text{offered}|$. Reporting both avoids a pathological result in which a policy achieves perfect conditional SLO by rejecting nearly all traffic. This distinction follows the goodput-oriented perspective used in modern SLO-aware serving [5], [11], [15].

IV. SYSTEM DESIGN

A. Production-Inspired Gateway

Our AWS deployment separates the production access path from the controlled evaluation path. In production, spoke AWS accounts call a central API Gateway using IAM SigV4; the gateway maps the authenticated principal to application metadata in DynamoDB. In evaluation, Locust injects a logical tenant identifier so that 100 tenant behaviors can be reproduced in one AWS account. The gateway runs on ECS Fargate, shares quota state through ElastiCache Serverless, streams responses from Amazon Bedrock, and writes per-request JSONL events for offline analysis. Amazon Nova Micro is the managed backend. The evaluation does not claim that the configured gateway control budget is an AWS service quota.

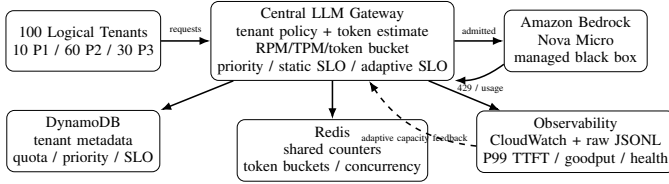


Fig. 1. Production-inspired architecture. The gateway observes tenant state and provider responses but not provider-side queues, batching, GPU occupancy, or instantaneous capacity.

B. Static Admission Policies

We implement six baselines. NONE forwards every request. RPM uses a calendar-minute request limit. TPM uses a calendar-minute token window. TOKEN-BUCKET continuously refills token credits and avoids minute-boundary reset artifacts. PRIORITY applies tier-aware allocation under pressure. STATIC-SLO combines token demand, tier, and remaining TTFT slack with capacity bands: below 80% budget requests are admitted; from 80–100% decisions become priority/SLO aware; from 100–110% only reserved shares are admitted; and at or above 110% the policy sheds.

C. Adaptive SLO-Aware Admission

The adaptive controller replaces the fixed platform budget with an effective capacity estimate $\hat{C}(t)$, initialized at the configured budget C_0 and bounded by C_{min} and C_{max} . In our deployment $C_0 = 100,000$ TPM, $C_{min} = 50,000$ TPM ($0.5 \times C_0$), and $C_{max} = 2,000,000$ TPM ($20 \times C_0$); the wide upper bound reflects the load-sweep finding (Section VI-A)

that the backend stayed healthy at $20 \times C_0$, so the controller is not artificially prevented from reaching that regime.

Every control window W (default 15 s), the controller reads window counters $admits$, $rejects$, slo_fail , and $bedrock_429$ and computes $offered = admits + rejects$, $reject_rate = rejects/offered$, and $fail_rate = slo_fail/admits$. We picked 15 s as a compromise: short enough that the controller adapts within a single 180-s burst phase (Section V), but long enough that a window typically accumulates the ≥ 5 samples our thresholds require, avoiding decisions driven by one or two requests. The update rule (Fig. 2) has three branches:

- **Decrease (provider pressure).** If $bedrock_429 > 0$, or $admits \geq 5$ and $fail_rate > 0.25$, the backend is treated as unhealthy and $\hat{C}$ is multiplicatively cut: $\hat{C} \leftarrow \max(C_{min}, \beta \hat{C})$ with $\beta = 0.7$. A 429 is an unambiguous provider-side signal; the SLO-fail branch is a secondary, looser-threshold signal so that a token-size jump alone (higher latency, no 429s) does not by itself collapse $\hat{C}$.
- **Increase-on-demand (healthy but shedding).** If $bedrock_429 = 0$ and $rejects \geq 5$ and $reject_rate \geq 0.2$, the gateway is shedding a meaningful share of traffic while the provider shows no distress; this is interpreted as $\hat{C}$ being under-budgeted, and it is raised multiplicatively: $\hat{C} \leftarrow \min(C_{max}, (1 + \alpha)\hat{C})$ with $\alpha = 0.15$.
- **Slow probe.** Otherwise $\hat{C}$ is nudged upward by a small additive step αC_0 toward C_{max} (a cautious default increase when neither pressure nor material shedding is observed, so the estimate keeps testing for headroom rather than sitting fixed).

Importantly, adaptation changes only the global capacity estimate; it does not eliminate tenant quotas or priority metadata, which continue to shape admission decisions given the current $\hat{C}$. This separation becomes central in the noisy-neighbor results: recovering globally available capacity is not equivalent to isolating a single aggressive tenant.

The controller is intentionally simple: one global state variable $\hat{C}$ and $O(1)$ work per request plus one $O(1)$ control update per window. This makes the policy suitable for an enterprise gateway data path and keeps the provider-facing adaptation interpretable. We do not claim $\hat{C}$ estimates physical GPU capacity; it represents a safe forwarding budget inferred from externally observable service behavior. We also do not claim novelty in the AIMD update itself [22]; the contribution is applying it to a capacity signal that is otherwise invisible to the gateway, and measuring the resulting isolation cost (Section VI-E).

Per-tenant TPM/RPM limits and priority weights are the entitlement inputs to every static and adaptive policy; only the platform-level capacity estimate (C_0 or $\hat{C}(t)$) differs across policies. Token demand per request follows the workload’s prompt class (Section V): 500 input / 64 max output tokens in the load-sweep and burst-normal phases, and 5000 input / 256 max output tokens during the burst window, applied uniformly

Require: $\hat{C} \leftarrow C_0$; $C_{min} = 0.5C_0$; $C_{max} = 20C_0$; $\alpha = 0.15$; $\beta = 0.7$; window $W = 15s$

- 1: Every window W : read *admits*, *rejects*, *slo_fail*, *bedrock_429* for the previous window
- 2: $reject_rate \leftarrow rejects / (admits + rejects)$;
 $fail_rate \leftarrow slo_fail / admits$
- 3: **if** *bedrock_429* > 0 **or** (*admits* $\geq$ 5 **and** *fail_rate* > 0.25) **then**
- 4: $\hat{C} \leftarrow \max(C_{min}, \beta\hat{C})$ *// provider pressure*
- 5: **else if** *bedrock_429* = 0 **and** *rejects* $\geq$ 5 **and** *reject_rate* $\geq$ 0.2 **then**
- 6: $\hat{C} \leftarrow \min(C_{max}, (1 + \alpha)\hat{C})$ *// healthy but shedding*
- 7: **else**
- 8: $\hat{C} \leftarrow \min(C_{max}, \hat{C} + \alpha C_0)$ *// slow probe*
- 9: **end if**
- 10: **for** each request r **do**
- 11: compute tenant quota, priority, token demand, and SLO slack using current $\hat{C}$
- 12: **return** ADMIT / QUEUE / SHED
- 13: **end for**

Fig. 2. Pseudocode for the adaptive SLO-aware capacity update.

TABLE I
EXPERIMENTAL PLATFORM AND TENANT CONFIGURATION

Component	Configuration
Gateway	ECS Fargate; streaming Bedrock client
State	DynamoDB tenant policy + shared Redis counters
Backend	Amazon Bedrock, Nova Micro
Tenants	100 logical: 10 P1 / 60 P2 / 30 P3
SLOs	P1 1 s; P2 2 s; P3 5 s TTFT
Policies	none, RPM, TPM, token bucket, priority, static-SLO, adaptive-SLO
Primary metrics	effective SLO goodput, conditional SLO, P99 TTFT, admission, throttle

TABLE II
PER-TIER TENANT CONFIGURATION (100 TENANTS)

Parameter	P1	P2	P3
Tenants	10	60	30
TPM limit	150,000	20,000	12,000
RPM limit	100	40	20
Max concurrency	10	4	2
TTFT SLO	1000 ms	2000 ms	5000 ms
E2E SLO	4000 ms	8000 ms	20,000 ms
Priority weight	4	2	1

across tiers so that burst pressure is a demand-side shock rather than a tier-specific effect.

V. EXPERIMENTAL METHODOLOGY

A. Workloads

The workload generator uses 100 logical enterprise tenants rather than 100 AWS accounts. Tenant profiles differ in tier, RPM/TPM entitlement, concurrency, prompt class, and TTFT SLO. This lets the experiment isolate admission behavior without confounding cross-account administration. We evaluate three workload families.

Load sweep varies offered load relative to the configured gateway control budget C_0 , using NONE to characterize whether C_0 resembles actual backend capacity. Token burst keeps request rate approximately constant while increasing token demand during a 180–360 s burst window; this specifically separates request-count pressure from token pressure. Noisy neighbor increases one tenant’s request rate by 10 $\times$ while reporting metrics over the other 99 tenants. For policy comparisons we freeze the latest five repetitions per policy; the exploratory load sweep has one repetition per load point.

B. Metrics and Measurement

TTFT is measured from gateway request start to the first streamed content event, not end-to-end completion. Each event records tenant, tier, estimated and actual token usage, admission decision, queue delay, TTFT, end-to-end latency, provider 429/5xx status, and SLO outcome. Effective SLO goodput is the primary service metric because it penalizes unnecessary rejection. We also report conditional SLO, P99 TTFT, admission rate, and gateway throttle rate. For noisy-neighbor analysis, the offending tenant is excluded from victim metrics.

C. Run Selection and Statistical Reporting

For policy comparisons, each headline result is computed from a frozen set of the latest five repetitions for that policy and scenario. We report arithmetic means across runs; where run-level dispersion is available we show mean $\pm$ sample standard deviation rather than implying precision from pooled request counts. The exploratory load sweep is explicitly treated separately because it has one repetition per load point. We avoid significance claims for the sensitivity ablation because alternate parameter settings have one run each. This protocol prevents older pilot runs from being mixed with the final experiment set and makes the comparison unit the independent run rather than the individual request.

The main effect sizes are large enough to interpret directly. In the token-burst phase, adaptive-SLO improves effective SLO goodput from 0.046 to 0.844 relative to static-SLO (18.3 $\times$), while P99 TTFT rises from 398 to 494 ms but remains below NONE at 566 ms. In noisy-neighbor runs, adaptive-SLO improves non-victim goodput from 0.847 to 0.997 while P99 increases from 394 $\pm$ 13 to 624 $\pm$ 88 ms (mean $\pm$ sample SD). We therefore emphasize effect size and tradeoff, not a universal winner.

D. Research Questions

RQ1: How well does a static gateway token budget represent usable capacity of a managed LLM backend? RQ2: How do static request-, token-, priority-, and SLO-aware controls trade effective goodput for tail latency? RQ3: Can adaptive capacity estimation recover goodput without surrendering SLO behavior under token-pressure bursts? RQ4: Does global adaptive capacity estimation also preserve tenant isolation under a noisy neighbor?

VI. RESULTS

A. RQ1: Static Gateway Budget Is Not Provider Capacity

Fig. 3 characterizes the uncontrolled backend over offered loads from 50% to 2000% of C_0 . Effective SLO goodput remains 0.998–1.000 and no provider throttling is observed. P99 TTFT ranges from approximately 385 to 519 ms. At 2000% of the configured gateway budget, effective SLO goodput is 0.999 and P99 TTFT is 472 ms. Because each point has one repetition, we treat this as exploratory characterization rather than a statistically replicated performance claim. Nevertheless, it establishes the key motivation: C_0 is an enterprise control budget, not a measurement of instantaneous Bedrock capacity.

B. RQ2: Static Admission Trades Goodput for Tail Latency

Under the noisy-neighbor workload, NONE and RPM retain essentially all non-victim effective goodput (0.999 and 1.000), with P99 TTFT of 562 and 529 ms. More aggressive static policies reduce P99 at the cost of shedding. STATIC-SLO lowers P99 to 394 ms but admits 86.7% of non-victim requests and yields 84.7% effective goodput. PRIORITY reaches a similar 397-ms P99 with 83.3% effective goodput. Thus, under this workload the main effect of static SLO-aware control is an explicit latency–admission tradeoff rather than a clear dominance in SLO success.

TABLE III
NOISY-NEIGHBOR RESULTS (NON-VICTIM TENANTS, 5 RUNS)

Policy	Admit	Eff. goodput	P99 TTFT	Throttle
None	1.000	0.999	562 ms	0.000
RPM	1.000	1.000	529 ms	0.000
Token bucket	0.914	0.910	451 ms	0.086
Priority	0.837	0.833	397 ms	0.163
Static SLO	0.867	0.847	394 ms	0.133

C. RQ3: Adaptive Control Recovers Token-Burst Goodput

The constant-RPM token burst exposes the central failure mode of static budgets. NONE and RPM both admit all requests and achieve 100% effective goodput, indicating that token demand rises sharply without saturating the provider. Static token-aware policies nevertheless interpret demand relative to C_0 as overload and shed 87–95% of burst traffic. STATIC-SLO obtains the lowest P99 among the static policies (398 ms), but its effective SLO goodput collapses to 4.6% because it admits only 4.9% of offered requests.

ADAPTIVE-SLO changes this outcome. During the same burst phase it admits 86.0% of requests and achieves 84.4% effective SLO goodput—approximately $18\times$ the static-SLO result—while conditional SLO among admitted requests improves to 98.2%. Its P99 TTFT is 494 ms: about 24% above static-SLO, but still below the 566-ms uncontrolled result. This is the main positive result: adaptation recovers most of the goodput that a fixed budget leaves on the table while retaining a meaningful share of the tail-latency benefit.

The run-level controller trace in Fig. 5 shows the corresponding closed-loop behavior. Once the effective budget reaches its configured upper bound, the policy largely stops budget-driven shedding; remaining admission variation is driven by tenant-level constraints and instantaneous workload state. TTFT spikes remain transient and provider 429s are absent in the representative run. We use this trace as a mechanism check rather than as an additional aggregate performance claim.

TABLE IV
TOKEN-BURST PHASE RESULTS (180–360 s, 5 RUNS)

Policy	Admit	Eff. goodput	Cond. SLO	P99
None	1.000	1.000	1.000	566 ms
RPM	1.000	1.000	1.000	532 ms
TPM	0.060	0.037	0.610	465 ms
Token bucket	0.113	0.073	0.646	511 ms
Priority	0.134	0.115	0.855	413 ms
Static SLO	0.049	0.046	0.949	398 ms
Adaptive SLO	0.860	0.844	0.982	494 ms

D. Controller Sensitivity

A small sensitivity study varies α , β , and control-window duration. Burst-phase effective goodput remains between 0.806 and 0.921 across five alternate configurations, versus 0.046 for static-SLO. Conditional SLO remains 0.970–0.989. The study uses one run per alternate setting, so it does not establish an optimum, but it shows that the order-of-magnitude recovery is not tied to one lucky parameter tuple.

TABLE V
ADAPTIVE PARAMETER SENSITIVITY (TOKEN BURST)

α	β	Window	Eff. goodput	P99
0.15	0.7	15 s	0.844	494 ms
0.05	0.7	15 s	0.833	476 ms
0.30	0.7	15 s	0.808	557 ms
0.15	0.5	15 s	0.921	535 ms
0.15	0.7	5 s	0.806	505 ms
0.15	0.7	30 s	0.866	552 ms

E. RQ4: Capacity Adaptation Is Not Tenant Isolation

The noisy-neighbor adaptive experiment provides a useful negative result. Adaptive-SLO essentially stops global shedding: non-victim admission is 1.000 and effective goodput is 0.997, compared with 0.867 admission and 0.847 effective goodput for static-SLO. However, non-victim P99 rises from

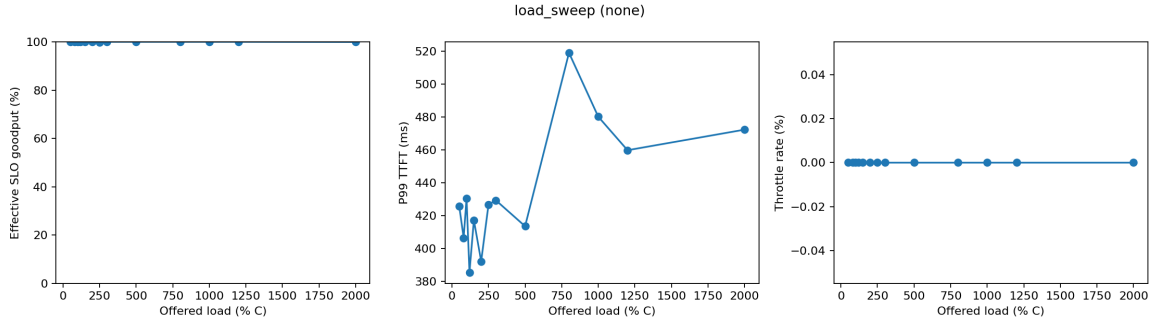


Fig. 3. Exploratory load sweep with no admission control. The managed backend remains healthy far above the configured gateway budget; “2000%” is relative to C_0 , not an AWS quota.

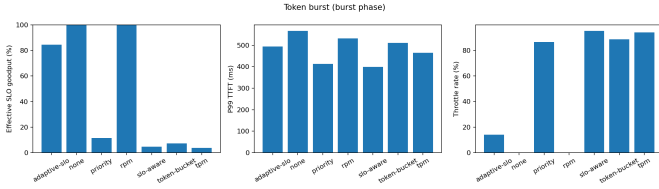


Fig. 4. Token-burst tradeoff. Adaptive SLO-aware admission recovers effective goodput while retaining lower P99 TTFT than uncontrolled admission.

394 to 624 ms. The tight-SLO P1 tier is affected most strongly: P1 P99 rises from 353 to 1811 ms, even though adaptive admission remains 100%. Thus a global controller that correctly learns that Bedrock has spare capacity can still fail to isolate tenants from a localized burst.

This result separates two control objectives that are easy to conflate. Capacity adaptation asks whether the provider can safely accept more aggregate work. Tenant isolation asks whether one tenant should be allowed to consume that work when doing so harms another tenant’s tail latency. *Adaptive provider-capacity estimation improves utilization, but tenant isolation requires a separate control mechanism.* A production controller therefore needs both: an adaptive global capacity estimate and an independent per-tenant protection mechanism such as reserved shares, burst debt, per-tenant caps, or a nonzero shedding floor. We view this negative result as one of the paper’s contributions rather than a limitation to downplay: it shows precisely where a single scalar feedback loop stops being sufficient.

Fig. 7 summarizes both scenarios on a single latency-goodput axis. In the token-burst panel, adaptive-SLO is the only policy other than the uncontrolled baselines (none, rpm) to reach high goodput, and it does so at lower P99 than either. In the noisy-neighbor panel, adaptive-SLO reaches the highest goodput of any policy but is a clear outlier on the latency axis — the visual signature of the isolation cost quantified in Table VI.

TABLE VI
ADAPTIVE VS. STATIC UNDER NOISY NEIGHBOR (NON-VICTIMS)

Policy	Admit	Eff. goodput	Cond. SLO	P99
Adaptive SLO	1.000	0.997	0.997	624±88 ms
Static SLO	0.867	0.847	0.977	394±13 ms

VII. DISCUSSION

A. Token Pressure Is Not Provider Saturation

The strongest empirical lesson is not that RPM is universally insufficient, but that request rate and token demand are different signals—and neither alone identifies saturation of an opaque elastic provider. In our token-burst experiment, RPM does not react because request count does not change, yet this is not a failure: the backend remains healthy. Conversely, static token-aware policies do react, but they react too strongly. Token accounting is still necessary for tenant governance, budgeting, and fairness [9], [16], [25], [26]; it is simply insufficient as a standalone estimate of instantaneous managed-provider capacity.

B. Why Adaptive Admission Helps

The adaptive controller closes the loop between pre-execution policy and post-execution evidence. The gateway initially treats the configured budget as a conservative prior. When it observes policy-driven rejects while TTFT remains stable and provider throttling is absent, it increases the effective capacity estimate. This approach resembles feedback-based congestion avoidance [22], but the goal is not to discover a fixed physical bottleneck. Instead, it tracks a time-varying safe operating region whose boundaries are set by a managed service.

A natural objection is that this is simply TCP-style congestion control relocated to an LLM gateway, and that AIMD’s convergence properties are already well understood. We agree, and do not claim novelty in the update rule itself. The novelty is not AIMD convergence, but controlling an externally unobservable, time-varying managed-service capacity using only black-box feedback (429s, TTFT, policy rejects) — signals several abstraction layers removed from any physical resource,

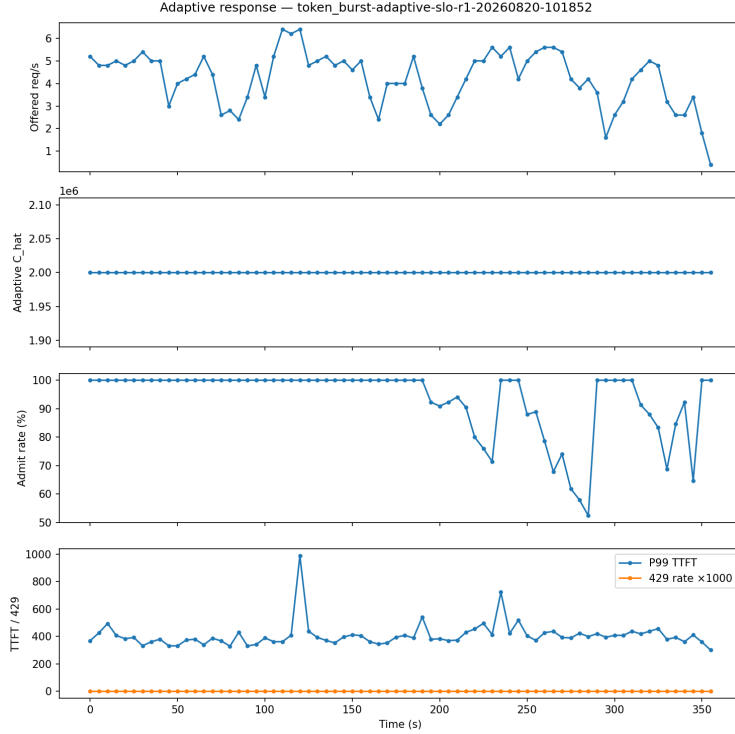


Fig. 5. Representative adaptive token-burst trace (r1): offered load, $\hat{C}(t)$, admit rate, and P99 TTFT / 429 rate over time. Once $\hat{C}(t)$ reaches its configured ceiling, budget-driven shedding stops and remaining admission variation reflects tenant-level constraints. The figure is used to verify controller dynamics; aggregate claims use the five-run summaries in Table IV.

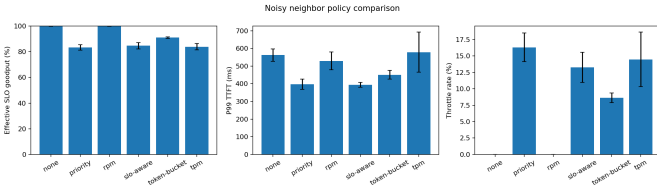


Fig. 6. Noisy-neighbor policy comparison across effective goodput, P99 TTFT, and throttle rate for non-victim tenants.

unlike TCP’s direct queue-loss signal. The empirical question this paper answers is whether that substitute is adequate, and what it costs when it is not (Section VI-E); that the answer is qualified rather than unconditionally positive is, in our view, the more useful contribution.

C. A Two-Level Control Boundary

The results suggest that aggregate capacity estimation and tenant isolation should not be fused into one scalar pressure threshold. A global AIMD loop can correctly infer that the provider has spare capacity while simultaneously allowing an aggressive tenant to increase another tenant’s tail latency. Conversely, a static tenant cap can provide isolation while leaving provider capacity unused. A more complete controller would therefore combine a provider-level adaptive budget with per-tenant mechanisms such as reserved shares, burst debt, weighted fair queuing, or a minimum shedding floor.

This decomposition is consistent with fairness-oriented serving work [9], [10], [16] but moves provider-capacity estimation outside the inference runtime.

D. Implications for Enterprise AI Platforms

A practical platform should separate at least three layers of control. First, provider-aware capacity adaptation determines how much aggregate work the gateway is willing to forward. Second, tenant-aware allocation determines who receives that capacity. Third, reliability logic handles provider 429s, retries, and cross-Region options when actual service limits are encountered [27], [32]. Treating these as separate concerns prevents an apparently healthy global backend from masking a noisy-neighbor failure.

E. Threats to Validity

Our evaluation uses one managed provider, one model family, one AWS region/control-plane configuration, and synthetic logical tenants. The load sweep is exploratory with one repetition per point. Provider capacity is intentionally not measured as a fixed AWS quota, and absence of observed 429s does not imply unlimited Bedrock capacity. Results may differ by model, region, account quota, provider-side load, and prompt distribution. The sensitivity study uses one repetition for alternate parameter settings. Finally, the adaptive policy currently uses global backend health and therefore does not fully solve tenant isolation; the noisy-neighbor result makes

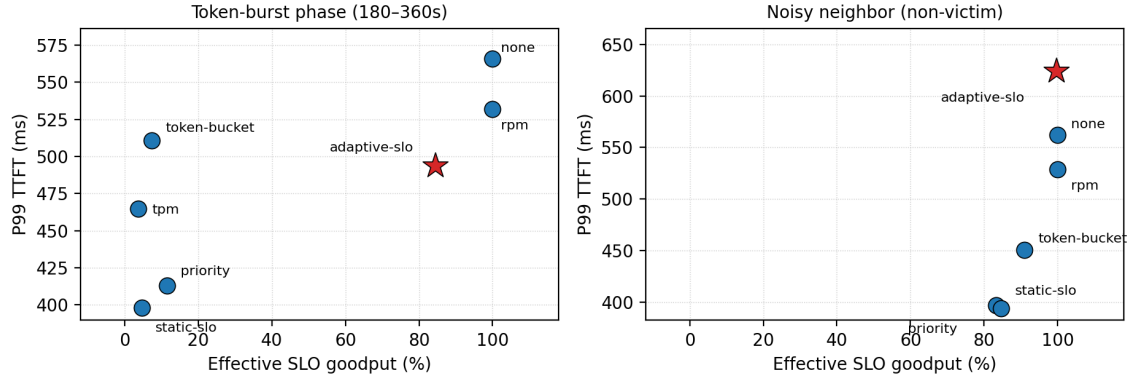


Fig. 7. Latency–goodput tradeoff across all seven policies, token-burst phase (left) and noisy-neighbor (right). Adaptive-SLO (star) sits near the goodput-maximizing edge in both scenarios; the noisy-neighbor panel makes the isolation cost visible as a P99 TTFT outlier rather than a goodput loss.

this limitation explicit. The public repository records workload definitions, selected run identifiers, raw-result analysis code, and frozen paper summaries so that the evaluation can be reproduced against future Bedrock conditions, although provider-side behavior itself may vary over time.

VIII. CONCLUSION

Multi-tenant gateways for managed LLM APIs face a control problem that differs from GPU-visible inference serving: the gateway must enforce tenant policy while the backend’s instantaneous capacity is opaque and elastic. Our Bedrock experiments show that a static token budget can be substantially more conservative than observed usable capacity. Static SLO-aware admission lowers tail latency but can severely over-shed traffic during token-demand bursts. Adaptive SLO-aware admission recovers burst-phase effective SLO goodput from 4.6% to 84.4% while retaining high conditional SLO attainment and lower P99 TTFT than uncontrolled admission. The noisy-neighbor experiment also establishes a boundary: global adaptation improves utilization but does not replace tenant isolation. These findings suggest a two-level design for production gateways—adaptive provider-capacity estimation coupled with explicit tenant-level protection—as a promising direction for reliable enterprise LLM platforms.

REFERENCES

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in Proc. 16th USENIX OSDI, 2022.
- [2] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in Proc. ACM SOSP, 2023.
- [3] B. Wu et al., “Fast distributed inference serving for large language models,” arXiv:2305.05920, 2023.
- [4] A. Agrawal et al., “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in Proc. 18th USENIX OSDI, 2024.
- [5] Y. Zhong et al., “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” arXiv:2401.09670, 2024.
- [6] P. Patel et al., “Splitwise: Efficient generative LLM inference using phase splitting,” in Proc. 51st ACM/IEEE ISCA, 2024, pp. 118–132.
- [7] R. Qin et al., “Mooncake: A KVCache-centric disaggregated architecture for LLM serving,” in Proc. 23rd USENIX FAST, 2025, pp. 155–170.
- [8] B. Sun et al., “Llumnix: Dynamic scheduling for large language model serving,” in Proc. 18th USENIX OSDI, 2024.
- [9] Y. Sheng et al., “Fairness in serving large language models,” arXiv:2401.00588, 2024.
- [10] R. I. S. Khan et al., “Ensuring fair LLM serving amid diverse applications,” arXiv:2411.15997, 2024.
- [11] Y. Tang, T. Lan, X. Huang, H. Lu, and W. Chen, “SCORPIO: Serving the right requests at the right time for heterogeneous SLOs in LLM inference,” arXiv:2505.23022, 2025.
- [12] K. Goel, J. Mohan, N. Kwatra, R. S. Anupindi, and R. Ramjee, “Niyama: Breaking the silos of LLM inference serving,” in Proc. ACM ASPLOS, 2026.
- [13] S. Chen, Z. Jia, S. Khan, A. Krishnamurthy, and P. B. Gibbons, “SLOs-Serve: Optimized serving of multi-SLO LLMs,” arXiv:2504.08784, 2025.
- [14] W. Huang et al., “ProServe: Unified multi-priority request scheduling for LLM serving,” arXiv:2512.12928, 2025.
- [15] M. Adnan et al., “Cascade: Exploiting SLO-aware latency budget for fair and high goodput LLM inference serving,” arXiv:2608.06557, 2026.
- [16] W. J. Cunningham, “Token management in multi-tenant AI inference platforms,” arXiv:2603.00356, 2026.
- [17] D. Crankshaw et al., “Clipper: A low-latency online prediction serving system,” in Proc. 14th USENIX NSDI, 2017.
- [18] A. Gujarati et al., “Serving DNNs like Clockwork: Performance predictability from the bottom up,” in Proc. 14th USENIX OSDI, 2020.
- [19] C. Zhang, M. Yu, W. Wang, and F. Yan, “MARK: Exploiting cloud services for cost-effective, SLO-aware machine learning inference serving,” in Proc. USENIX ATC, 2019.
- [20] D. Crankshaw et al., “InferLine: ML prediction pipeline provisioning and management for tight latency objectives,” in Proc. 11th ACM SoCC, 2020.
- [21] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, “INFaaS: Automated model-less inference serving,” in Proc. USENIX ATC, 2021.
- [22] D.-M. Chiu and R. Jain, “Analysis of the increase and decrease algorithms for congestion avoidance in computer networks,” Comput. Netw. ISDN Syst., vol. 17, no. 1, pp. 1–14, 1989.
- [23] J. Dean and L. A. Barroso, “The tail at scale,” Commun. ACM, vol. 56, no. 2, pp. 74–80, 2013.
- [24] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, “Sparrow: Distributed, low latency scheduling,” in Proc. ACM SOSP, 2013.
- [25] Amazon Web Services, “Amazon Bedrock quotas,” Amazon Bedrock User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/quotas.html>
- [26] Amazon Web Services, “Token burndown and runtime quotas for Amazon Bedrock,” Amazon Bedrock User Guide, 2026. [Online]. Available:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/quotas-token-burndown.html>
- [27] Amazon Web Services, “Increase throughput with cross-Region inference,” Amazon Bedrock User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>
 - [28] Amazon Web Services, “ConverseStream,” Amazon Bedrock Runtime API Reference, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ConverseStream.html
 - [29] Amazon Web Services, “Count tokens before model inference,” Amazon Bedrock User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/count-tokens.html>
 - [30] Amazon Web Services, “Monitor Amazon Bedrock runtime metrics with CloudWatch,” Amazon Bedrock User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>
 - [31] Amazon Web Services, “Token usage in Amazon Bedrock Runtime,” Amazon Bedrock Runtime API Reference, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_TokenUsage.html
 - [32] Amazon Web Services, “Retry behavior,” AWS SDKs and Tools Reference Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html>
 - [33] Amazon Web Services, “Throttle API requests for better throughput,” Amazon API Gateway Developer Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html>
 - [34] Amazon Web Services, “Elastic Load Balancing for Amazon ECS services,” Amazon ECS Developer Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/alb.html>
 - [35] Amazon Web Services, “Amazon ElastiCache components and serverless caches,” Amazon ElastiCache User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.corecomponents.html>
 - [36] Amazon Web Services, “IAM tutorial: Delegate access across AWS accounts using IAM roles,” AWS IAM User Guide, 2026. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_cross-account-with-roles.html
 - [37] J. Duan et al., “MuxServe: Flexible spatial-temporal multiplexing for multiple LLM serving,” arXiv:2404.02015, 2024.
 - [38] A. Prasad et al., “Preble: Efficient distributed prompt scheduling for LLM serving,” arXiv:2407.00023, 2024.
 - [39] R. Jain, D.-M. Chiu, and W. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” DEC Research Report TR-301, 1984.
 - [40] Amazon AGI, “The Amazon Nova family of models: Technical report and model card,” arXiv:2506.12103, 2025.